AI Assistant App for Plant Disease Diagnosis

(Project Proposal)

Project Code

FYP2226G

Project Advisor

Dr. Hussam Ali

Project Manager

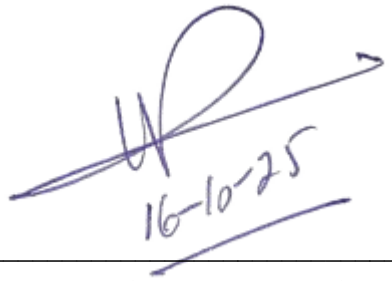
Prof. Dr. Muhammad Iliyas

Project Team

Muhammad Taha Mukhtar BSCS51F22S004 (Team Lead)
Hammad Ahmad BSCS51F22S012 (Team Member)
Usama Farooq BSCS51F22S045 (Team Member)

Submission Date

16/10/25



Project Manager's Signature

Table of Contents

Abstract.....3
Background and Justification.....3
Project Methodology.....3
Project Scope.....4
High level Project Plan.....4
References..... 6

Abstract

Farmers and home gardeners often face issues like crop diseases, unpredictable weather, and limited access to expert advice. AgroGuard is a mobile app that uses AI to solve these problems by combining essential farming tools in one easy-to-use platform. The app allows users to take pictures of plant leaves to identify diseases using a machine learning model. It also gives treatment suggestions created by an AI language model. A built-in chatbot enables users to ask farming questions in everyday language and get useful answers. The app provides real-time weather updates based on the user's location and shares the latest agricultural news to keep farmers informed. AgroGuard is designed for Android, supports various crop types, and uses cloud-based services for login and data storage. It is made simple enough for both seasoned farmers and casual gardeners to use without any technical skills. This project demonstrates how AI and mobile technology can work together to make farming smarter and more accessible. By helping users spot crop problems early and take appropriate action, AgroGuard seeks to cut down crop losses and promote more productive and sustainable farming practices.

Background and Justification

Agriculture is essential for food security and the economy, but many farmers still face problems like unpredictable weather, crop diseases, and limited access to expert advice. Most rely on traditional methods, which often lead to low productivity and financial losses. Although AI-based tools for farming, such as plant disease detection and weather prediction, already exist, they are usually limited to specific functions and are not always easy for farmers to use [1], [3].

This project aims to improve and expand on those existing solutions by developing AgroGuard, a user-friendly AI-powered Farmer Assistant App. Unlike prior solutions that rely on on-device processing, AgroGuard uses a client-server architecture where machine learning models are hosted on a remote server and accessed through an API. This eliminates large app sizes, enables seamless model updates without releasing a new version of the app, and reduces the processing load on the user's device. The app combines plant disease detection, AI-generated treatment advice, real-time weather updates, agricultural news, and an AI chatbot, all in one platform [1], [2].

The goal is to make advanced agricultural technology accessible to all farmers, especially those with little technical knowledge. By offering accurate, real-time insights in one platform, this system will help farmers make better decisions, boost productivity, and adapt more effectively to changing environmental conditions [3], [4].

Project Methodology

The first step is data collection, which plays a vital role in building accurate and reliable AI models. Plant disease image datasets are sourced from publicly available agricultural datasets containing thousands of labeled images across multiple crop species. Additionally, a custom dataset of locally

collected crop images is compiled by the project team to improve regional accuracy and better represent the target environment.

For the disease detection feature, machine learning models are trained for each supported crop type and deployed on a remote server. Each crop has a dedicated endpoint that accepts an image and returns a response containing the detected disease name and confidence score.

Before sending an image to the server, the app performs on-device image processing to isolate the plant leaf from the background. If this process is unsuccessful, a fallback method is applied to ensure the image is still usable. The processed image is then compressed and uploaded to the server for classification.

Post-diagnosis treatment advice is generated by an AI language model, producing chemical suggestions, fertilizer recommendations, and a practical action plan. The in-app chatbot also uses the same AI service, maintaining conversation context to provide relevant and consistent responses. Weather data is fetched based on the farmer's GPS location, delivering key information such as temperature, humidity, and wind speed. An agricultural news feed is also integrated to keep users updated with the latest farming-related news.

Once all features are integrated, the application brings them together through a clean and user-friendly interface, allowing farmers to easily access information, receive advice, and interact with the AI chatbot through natural language.

Project Scope

Included Features:

- Plant disease detection for six crop types (Apple, Orange, Lemon, Corn, Potato, and Mango) through machine learning models hosted on a remote server. The app sends preprocessed images and receives the detected disease name along with a confidence score.
- On-device image processing to isolate the plant leaf from the background before sending the image to the server for classification.
- AI-generated treatment advice including chemical suggestions, fertilizer recommendations, and a practical action plan, displayed to the user after a disease is detected.
- Real-time weather updates based on the user's location, providing key information such as temperature, humidity, and wind speed to help farmers plan their watering, harvesting, and spraying schedules.
- An agricultural news feed displaying current global farming and crop-related articles to keep users informed of the latest developments in agriculture.
- A smart AI chatbot that allows farmers to ask questions and receive instant, relevant farming advice through natural language conversation.

- User authentication supporting both email/password login and third-party sign-in, along with profile management through a cloud-based platform.
- The system aims to make farming smarter, faster, and more efficient by bringing all these tools together in one accessible mobile application.
- Overall, the application helps farmers protect their crops, save time, and increase productivity by providing the right information at the right time.

Not Included:

- The system does not include advanced hardware-based tools such as drones or sensors for field monitoring.
- Automated systems for irrigation, pest control, or fertilizer application are not part of this phase of development.
- All disease classification is performed on a remote server rather than directly on the user's device.
- The application is currently limited to a single mobile platform and does not support other operating systems or desktop environments.
- These features and expansions may be considered and added in future versions as the project continues to grow and develop.

High level Project Plan

Task Name	Duration	Start	Finish	Description
Proposal Writing & Submission	7 days	01-Oct-2025	07-Oct-2025	Prepare and finalize the project proposal for approval.
Data Collection (Part 1)	7 days	08-Oct-2025	14-Oct-2025	Gather initial plant disease image dataset from PlantVillage and other public sources.
Data Collection (Part 2 & Preparation)	7 days	15-Oct-2025	21-Oct-2025	Continue data collection, clean and preprocess data. Collect local crop images for the custom dataset.
Prepare SRC Document	14 days	22-Oct-2025	04-Nov-2025	Prepare the Student Research Committee (SRC) documentation.
SRC Submission (Milestone)	2 days	05-Nov-2025	06-Nov-2025	Submit SRC documentation for review and approval.
Requirement Gathering	14 days	07-Nov-2025	20-Nov-2025	Identify functional and non-functional requirements from stakeholders.

System Architecture Design	14 days	21-Nov-2025	04-Dec-2025	Design MVVM architecture, Firebase integration, Hugging Face server plan, and Retrofit API layer.
Frontend Navigation	7 days	05-Dec-2025	11-Dec-2025	Develop and implement navigation components for the frontend interface.
Frontend Design	7 days	12-Dec-2025	18-Dec-2025	Create and integrate user interface layouts and visual design elements.
Design Document Finalization	6 days	19-Dec-2025	24-Dec-2025	Finalize and review design documentation before development.
Model Training	15 days	25-Dec-2025	08-Jan-2026	Train TFLite models for Apple, Orange, Lemon, Corn, Potato, and Mango on PlantVillage and custom datasets.
Model Deployment (Hugging Face)	7 days	09-Jan-2026	15-Jan-2026	Deploy trained TFLite models to a Hugging Face Space (Python FastAPI server) with crop-specific endpoints.
Backend Integration	14 days	16-Jan-2026	29-Jan-2026	Integrate Retrofit clients, Groq API, Firebase Auth, Firestore, and Storage.
Testing & QA	14 days	30-Jan-2026	12-Feb-2026	Unit and integration testing of ViewModels, API clients, and UI flows on multiple Android devices.
Final Documentation & Submission	7 days	13-Feb-2026	19-Feb-2026	Finalize all documents, record demo, and submit for FYP evaluation.

References

[1] P. S. Venkata Reddy, K. S. Nandini Prasad, and C. Puttamadappa, "Farmer's Friend: Conversational AI Bot for Smart Agriculture," Journal of Positive School Psychology, vol. 6, no. 2, pp. 2541–2549, 2022. Available: https://journalppw.com/index.php/jpsp/article/view/1833

[2] V. Nayak, N. H. Sowmya, et al., "AgroXpert: Farmer Assistant," Global Transitions Proceedings, vol. 2, no. 2, pp. 506–512, 2021. Available: https://www.scribd.com/document/716551511/Agroxpert-Farmer-assistant-2021-Global-Transitions-Proceedings

[3] A. Jafar, N. Bibi, R. A. Naqvi, A. Sadeghi-Niaraki, and D. Jeong, "Revolutionising Agriculture with Artificial Intelligence: Plant Disease Detection Methods, Applications, and Their Limitations," Frontiers in Plant Science, vol. 15, Article 1356260, 2024. Available: https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2024.1356260/full

[4] T. Yadav, P. Sable, and D. Kalbande, "Smart Kisan: A Mobile App for Farmers's Assistance in Agricultural Activities," in Proceedings of the 2023 International Conference on Smart Systems for Applications in Electrical Sciences (ICSSES), 2023, pp. 1–6.

Software Requirements Specifications

AI Assistant App for Plant Disease Diagnosis

Project Code: FYP2226G

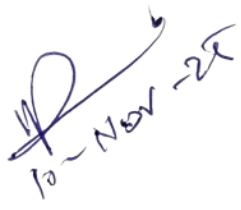
Internal Advisor: Dr. Hussam Ali

External Advisor: N/A

Project Manager: Prof. Dr. Muhammad Iliyas

Project Team: M Taha Mukhtar BSCS51F22S004 (Team Lead)
Hammad Ahmad BSCS51F22S012 (Team Member)
Usama Farooq BSCS51F22S045 (Team Member)

Submission Date: 10th Nov 2025



Project Manager's Signature

Document Information

Category	Information		
Customer	University of Sargodha	Project	AI Assistant for Plant Disease Diagnosis (Agro Guard)
Document	Software Requirements Specification	Document Version	4.0
Identifier	FYP2226G	Status	Approved Version
Author(s)	M Taha Mukhtar, Hammad Ahmad, Usama Farooq	Approver(s)	Prof. Dr. Muhammad Iliyas
Issue Date	OCT 20th, 2025		

Document Location Distribution

1. Advisor 2. PM 3. Project Office

Definition of Terms, Acronyms and Abbreviations

Term	Description
AI	Artificial Intelligence
API	Application Programming Interface
XML	Extensible Markup Language
JSON	JavaScript Object Notation
GPS	Global Positioning System
ML	Machine Learning
PNG	Portable Network Graphics
JPEG	Joint Photographic Expert Group
HF	Hugging Face
TFLite	TensorFlow Lite
LLM	Large Language Model
MVVM	Model-View-ViewModel

Table of Contents

1. Introduction 4
1.1 Purpose of Document 4
1.2 Project Overview 4
1.3 Scope Included Features 5
2. Overall System Description 6
2.1 User Characteristics 6
2.2 Operating Environment 6
2.3 System Constraints 6
3. External Interface Requirements 8
3.1 Hardware Interfaces 8
3.2 Software Interfaces 8
3.3 Communications Interfaces 9
4. Functional Requirements 10
5. Non-Functional Requirements 12
5.1 Performance Requirements 12
5.2 Safety Requirements 12
5.3 Security Requirements 12
6. Assumptions and Dependencies 13
7. References 14
8. Appendices 15
8.1 Dataset Description 15

1. Introduction

1.1 Purpose of Document

This document describes the functionality, objectives, and scope of the Plant Disease Detection System (AgroGuard). It describes how the system works, its intended users, and the benefits it aims to deliver. This document is intended for:

1. **Developers:** To understand the system's architecture, functionality, and technical requirements.
2. **Project Supervisors/Stakeholders:** To evaluate the project's objectives, scope, and expected outcomes.
3. **End Users (Farmers):** To understand how the system assists in diagnosing plant diseases and managing crop health efficiently.

1.2 Project Overview

AgroGuard is an AI-powered mobile application developed using Kotlin and Jetpack Compose for Android. The system is designed to assist farmers in identifying plant diseases through image recognition and provide effective management recommendations.

Key Features

1. **AI-Based Disease Detection:** Farmers can capture or upload images of their plants. The image is preprocessed on-device using ML Kit Subject Segmentation to isolate the leaf, then sent to a remote TFLite inference server hosted on Hugging Face for disease classification. Results include the disease name and confidence score along with treatment recomendations.
2. **Disease Information and Treatment Suggestions:** Upon detection, the system queries the Groq API (Llama 3.3 70B Versatile model) to generate expert-level treatment advice including chemical suggestions, fertilizer recommendations, and an immediate action plan.
3. **Weather Updates:** Offers live weather information using the device's GPS and the OpenWeatherMap API, displaying temperature, humidity, wind speed, and conditions on the home dashboard.
4. **Agricultural News:** Displays a real-time feed of agriculture-related news articles fetched from NewsAPI, covering global trends in farming and crop management.

5. **AI Chatbot (AgroBot):** An intelligent conversational agent powered by the Groq API (Llama 3.3 70B Versatile) that allows farmers to ask questions in natural language about crop care, disease prevention, and general agriculture.
6. **User Authentication and Profile Management:** Supports email/password registration and Google Sign-In via Firebase Authentication. Users can manage their profile, including a profile photo stored in Firebase Storage.
7. **User-Friendly Interface:** The application provides a simple, intuitive interface built with Jetpack Compose, featuring a bottom navigation bar with four tabs: Home, Diagnose, Assistant, and Profile.

Goals and Benefits

1. Enhance crop health through early and accurate disease detection.
2. Reduce crop losses caused by unidentified or untreated diseases.
3. Minimize dependency on agricultural experts by providing AI-driven recommendations.
4. Empower farmers with accessible, reliable, and quick diagnostic support.

1.3 Scope

Included Features

1. Image-based detection of plant diseases using Artificial Intelligence via a remote Hugging Face-hosted TFLite inference server.
2. Support for six crop types: Corn, Tomato, Potato, Apple, Orange, and Rice.
3. Display of disease-related information and suitable treatment or preventive measures generated by Groq API.
4. Real-time weather updates relevant to agricultural activities based on GPS location (OpenWeatherMap API).
5. Agricultural news feed from NewsAPI.
6. An intelligent conversational agent (AgroBot) powered by Groq API (Llama 3.3 70B Versatile) for natural language agriculture queries.
7. User authentication (email/password and Google Sign-In) and profile management via Firebase.
8. Urdu language support for better understanding of the farmers, and those who are not familiar or comfortable with English.

Not Included

1. Advanced agricultural automation tools such as drones for field monitoring.
2. Automated irrigation, pest control, or fertilizer application systems.
3. Marketplace or product-selling functionalities.
4. On-device ML model inference (all inference is server-side).
5. iOS or desktop platform support.

2. Overall System Description

2.1 User Characteristics

AgroGuard is primarily designed for farmers and people involved in agriculture who want an easy way to identify plant diseases and get useful guidance.

Farmer:

Farmers are the main users of this app. Most of them may not have technical knowledge, so the system is designed with a simple and easy-to-understand interface. They can capture or upload pictures of their plants and get quick feedback about possible diseases and treatments from the AI system.

Agricultural Experts (optional users):

Experts or agricultural officers may use the system to verify results, provide recommendations, or use it as a support tool in research or advisory work.

The app focuses on being simple, reliable, and helpful for all users, even those with minimal technical experience.

2.2 Operating Environment

The system is developed as a mobile application that runs on Android devices.

Operating Details:

1. Platform: Android smartphones and tablets.
2. Programming Language: Kotlin.
3. UI Framework: Jetpack Compose (declarative UI).

4. Minimum SDK: API Level 26 (Android 8.0); Target SDK: API Level 35 (Android 15).
5. Internet Requirement: The app requires a stable internet connection for image uploads to the Hugging Face inference server, AI chatbot (Groq API), weather updates, and news fetching. No offline disease detection is supported.
6. AI Inference: Disease detection is performed on a cloud-hosted TFLite model server (Hugging Face Space). The app sends a preprocessed image and receives a JSON response with the disease name and confidence score.

2.3 System Constraints

The following constraints and limitations may affect how the system operates:

Software Constraints:

1. The app is currently developed for Android only; it is not available for iOS or desktop platforms.
2. Requires active internet access for disease detection, chatbot, weather, and news features.
3. The AI model's accuracy depends on the quality and clarity of the uploaded plant images. Leaf segmentation via ML Kit is applied on-device to improve isolation, but extremely poor images may yield uncertain results.
4. The app supports six crop types: Corn, Tomato, Potato, Apple, Orange, and Rice. Other crops are not supported in the current version.

Hardware Constraints:

1. Users need a smartphone with a camera and internet connectivity.
2. The device should have sufficient memory (minimum 2GB RAM) to run the app and handle image processing.
3. Minimum 50MB free storage for app installation (no on-device ML model is bundled).

Legal Constraints:

1. Users must only upload images they own or have permission to use.

Environmental Constraints:

1. The app is designed for outdoor use, but performance may be affected by poor network signals in rural areas, as all ML inference requires an active internet connection.

Performance Constraint:

The app is designed to detect plant diseases from uploaded images under normal internet conditions. Network requests use a 60-second timeout (OkHttp). A loading indicator is displayed during image upload and inference.

3. External Interface Requirements

3.1 Hardware Interfaces

Mobile Device Requirements:

1. **Platform:** Android smartphones
2. **Minimum Android Version:** Android 8.0 (API 26) or higher
3. **RAM:** Minimum 2 GB; Recommended 4 GB or higher
4. **Storage:** Minimum 50 MB free space for app installation (no ML model bundled)
5. **Camera:** Rear camera with minimum 8 MP resolution for plant disease image capture
6. **Network:** Wi-Fi or mobile data connectivity (3G/4G/5G) for server communication

GPS/Location Services:

1. The application accesses the device's GPS hardware (via Android's FusedLocationProviderClient) to obtain location coordinates.
2. Location information is used to provide localized weather updates via OpenWeatherMap API.

3.2 Software Interfaces

Mobile Application Platform:

1. **Development Framework:** Kotlin programming language with Jetpack Compose
2. **Target Platform:** Android OS
3. **IDE:** Android Studio
4. **Minimum SDK:** API Level 26 (Android 8.0)
5. **Target SDK:** API Level 35 (Android 15)
6. **UI Framework:** Jetpack Compose (declarative UI toolkit)

External APIs and Services:

Hugging Face Inference Server (Disease Detection):

1. **Purpose:** Perform TFLite-based plant disease classification
2. **Interface:** Multipart HTTP POST via Retrofit + OkHttp
3. **Input:** JPEG image sent as multipart/form-data
4. **Output:** JSON response with disease name and confidence score
5. **Timeout:** 60 seconds (connect, read, write)

Groq API (AI Chatbot and Treatment Advice):

1. **Purpose:** Power the AgroBot chatbot and generate post-diagnosis treatment advice
2. **Model:** llama-3.3-70b-versatile
3. **Interface:** HTTPS POST via Retrofit, JSON body with message history
4. **Authentication:** Bearer token (API key from BuildConfig.GROQ_API_KEY)
5. **Data Exchange Format:** JSON

OpenWeatherMap API (Weather):

1. **Purpose:** Fetch real-time weather data based on user's GPS location
2. **Data Retrieved:** Temperature (°C), humidity (%), wind speed (km/h), weather condition
3. **Authentication:** API key (appid query parameter)
4. **Data Exchange Format:** JSON

NewsAPI (Agricultural News):

1. **Purpose:** Fetch current agriculture-related news article
2. **Data Exchange Format:** JSON
3. **Authentication:** API key (apiKey query parameter from BuildConfig.NEWS_API_KEY)

Firebase Services:

1. **Firebase Authentication:** Email/password registration and login; Google Sign-In via GoogleSignInOptions and GoogleAuthProvider
2. **Cloud Firestore:** NoSQL document database storing user profiles (users collection)

3.3 Communications Interfaces

User Interfaces:

1. **Home Screen:** Weather dashboard (temperature, humidity, wind speed, condition), quick action cards (AgroBot, News), camera launch button
2. **Diagnose Screen:** Crop selection grid (Corn, Tomato, Potato, Apple, Orange,Rice), image capture/upload, diagnosis trigger
3. **Camera Screen:** CameraX-based live camera preview with capture button
4. **Result Screen:** Displays disease name, confidence score, and AI-generated treatment advice (chemical suggestions, fertilizer recommendations, action plan)
5. **Assistant Screen (AgroBot):** Chat interface with message bubbles, timestamps
6. **News Screen:** Scrollable list of agricultural news cards with title, description, source, and image
7. **Profile Screen:** User info (name, email, profile photo), settings (Security, Privacy Policy), language Selection, Sign Out
8. **Edit Profile Screen:** Update name, phone number
9. **FAQ Screen:** Frequently asked questions about the app
10. **Login/Sign Up Screens:** Email/password fields and Google Sign-In button

4. Functional Requirements

4.1 Weather Information Display

1. The system shall display current weather information on the user's home screen dashboard, including temperature (°C), humidity (%), wind speed (km/h), and weather condition.
2. The system shall automatically detect the user's location using device GPS (FusedLocationProviderClient) to provide localized weather updates via the OpenWeatherMap API.

4.2 Plant Disease Detection

1. The system shall allow users to select a crop type (Rice,Corn, Tomato, Potato, Apple, or Orange) before initiating diagnosis.
2. The system shall allow users to capture a plant image using the built-in CameraX interface or upload one from the device gallery.
3. The system shall apply ML Kit Subject Segmentation on-device to isolate the leaf from the background before sending the image to the server.
4. The system shall send the processed image to the Hugging Face-hosted TFLite inference server via a multipart HTTP POST request using the appropriate crop endpoint.

5. The system shall display the detected disease name and confidence score returned from the server.
6. The system shall display an appropriate message if the image is unclear or the result is uncertain, prompting the user to retake the image.
7. The system shall display 'Healthy' if no disease is detected.

4.3 Disease Detection System (Detailed Workflow)

Image Preprocessing (On-Device):

1. The system shall apply ML Kit Subject Segmentation to remove the image background, using a green mask fallback (FallbackGreenMask) if segmentation fails.
2. The system shall compress the processed image to JPEG format before upload.
3. The system shall route the image to the correct crop endpoint based on the user-selected crop type.

AI Inference (Server-Side):

1. The server shall run the preprocessed image through the appropriate TFLite model.
2. The server shall return a JSON response containing the disease name (prediction field) and confidence score.

Results and Recommendations:

1. The system shall call the Groq API (Llama 3.3 70B) to generate treatment advice when a disease (non-healthy result) is detected.
2. The treatment advice shall be structured into three sections: Chemical Suggestions (fungicides/pesticides with dosages), Fertilizer Recommendations (NPK ratios and application rates), and Action Plan (2–3 immediate steps for the farmer).
3. The system shall display the full treatment advice on the Result Screen.

4.4 AI Chatbot Interaction (AgroBot)

1. The system shall provide a chat interface (AssistantScreen) accessible from the bottom navigation bar.
2. The system shall allow users to send text-based queries regarding crop care, disease prevention, and general agriculture.

3. The system shall generate responses using the Groq API (Llama 3.3 70B Versatile model), maintaining conversation context using the last 10 messages.
4. The system shall display bot responses with timestamps alongside user messages.

4.5 Agricultural News

1. The system shall display a feed of agriculture-related news articles fetched from NewsAPI on the News Screen.
2. Each news card shall display the article title, description, publication date, source name, and thumbnail image.
3. The system shall allow users to open the full article in a browser by tapping the news card.

4.6 User Authentication and Profile Management

1. The system shall support user registration via email/password and Google Sign-In using Firebase Authentication.
2. The system shall create a corresponding user document in Firestore upon successful registration.
3. The system shall allow users to view and edit their profile (name, phone number) via the Edit Profile screen.
4. The system shall store profile images in Firebase Storage.
5. The system shall support user logout, clearing the Firebase Auth session.

5. Non-Functional Requirements

5.1 Performance Requirements

1. The app shall be optimized to run efficiently on Android devices running Android 8.0 (API 26) or later, with minimal memory and CPU consumption.
2. Image preprocessing (ML Kit leaf segmentation and JPEG compression) shall be performed on-device.
3. All network requests (Hugging Face inference, Groq API, weather, news) shall be performed asynchronously using Kotlin Coroutines and ViewModel scope.
4. All Retrofit clients (RetrofitClient and GroqClient) shall be configured with 60-second connect, read, and write timeouts to handle server processing time.
5. The app shall display loading indicators (UiState.Loading) to the user during image upload and inference phases.

6. The app should be available 99.9% of the time for local features; availability of remote inference and API features depends on third-party server uptime.

5.2 Safety Requirements

1. The system shall ensure that no harmful actions occur due to invalid inputs, such as uploading non-image files or corrupted images. The DiagnosisViewModel handles null and error responses gracefully.
2. The app shall handle all exceptions (IOException, HttpException) gracefully, displaying clear error messages via UiState.Error without exposing internal stack traces.
3. The system shall prompt users to retake the image if the server returns an 'uncertain' prediction.
4. The system shall not make direct agricultural treatment recommendations without including appropriate disclaimers to avoid user harm.

5.3 Security Requirements

1. All communications between the app and remote services (Hugging Face, Groq API, OpenWeatherMap, NewsAPI, Firebase) shall use HTTPS encryption.
2. API keys (GROQ_API_KEY, NEWS_API_KEY, Weather API key) shall be stored securely in BuildConfig via local.properties and shall not be committed to version control.
3. Any temporary image files created during processing shall be stored in the app's cache directory and may be cleared by the OS; no permanent local storage of diagnosis images.
4. Firebase Authentication shall manage all user session tokens and refresh cycles automatically.
5. Firebase Firestore security rules shall protect user data from unauthorized access.
6. The app shall clearly state its data privacy policy on the Profile screen (Privacy Policy option).

6. Assumptions and Dependencies

Assumptions

1. Users will have a stable internet connection to upload images and receive predictions, as no on-device inference is supported.
2. Users' devices will have a functional camera and sufficient storage for temporary image handling during processing.

3. The TFLite models on the Hugging Face server have been pre-trained and optimized for the five supported crop types (Corn, Tomato, Potato, Apple, Orange) before deployment.
4. The project uses publicly available datasets (PlantVillage, AI Challenger) or proprietary data collected under consent for model training.
5. The Hugging Face Space hosting the inference server will remain active and accessible during the app's operational period.

Dependencies

1. Remote Inference Server: The application depends on a Hugging Face-hosted server running TFLite models for plant disease inference. No on-device ML framework (TFLite, ML Kit models) is used for inference.
2. AI Chatbot and Treatment: Groq API using the Llama 3.3 70B Versatile model for both the AgroBot assistant and post-diagnosis treatment text generation.
3. Weather Feature: Integration with OpenWeatherMap API (api.openweathermap.org) for real-time weather data.
4. News Feature: Integration with NewsAPI (newsapi.org) for agricultural news articles.
5. Firebase: Firebase Authentication, Cloud Firestore, and Firebase Storage for user management and data persistence.
6. Image Processing: ML Kit Subject Segmentation for on-device leaf isolation; CameraX for image capture.
7. Google Sign-In: com.google.android.gms:play-services-auth for Google OAuth integration.
8. Project success depends on accurate dataset labeling, model training quality, and continued availability of third-party APIs.

7. References

Ref. No.	Document Title	Date of Release/ Publication	Document Source
Group 40	Project Proposal	Oct 13, 2025	https://github.com/mtahamukhtar2-ui/FYP
[1]	PS Venkata Reddy et al. Farmer's	2022	https://journalppw.com/index.php/jpsp/article/view/1833

	Friend: Conversational AI Bot for Smart Agriculture. Journal of Positive School Psychology, 6(2):2541–2549		
[2]	Vandana Nayak et al. Agroxpert – Farmer Assistant. Global Transitions Proceedings, 2(2):506–512	2021	https://www.scribd.com/document/716551511/Agroxpert-Farmer-assistant
[3]	Abbas Jafar et al. Revolutionising Agriculture with Artificial Intelligence: Plant Disease Detection Methods. Frontiers in Plant Science, 15:1356260	2024	https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2024.1356260/full
[4]	Tejal Yadav et al. Smart Kisan: A Mobile App for Farmers' Assistance. IEEE ICSSES 2023	2023	IEEE Xplore (conference proceedings)

8. Appendices

8.1 Dataset Description

The TFLite models for plant disease detection were trained on trusted, publicly available datasets to ensure accurate results across the supported plant types (Corn, Tomato, Potato, Apple, Orange). The models are hosted on a Hugging Face Space and are not bundled with the Android application.

Dataset Name	Source	Description	Format
Plant Dataset	Kaggle, Mendeley	Contains over 54,000 labeled images.	JPEG
AI Challenger Dataset	AI Challenger Platform	Provides agricultural images for machine vision applications, used for cross-validation.	PNG
Custom-Collected Dataset	Project Team	Locally collected images of regional crops for enhanced model accuracy and real-world testing.	JPEG

Software Design Specification

Project Title:

AI Assistant App for Plant Disease Diagnosis
(AgroGuard App)

Project Code:

FYP2226G

Internal Advisor:

Dr. Hussam Ali

Externa Advisor:

N/A

Project Manager:

Prof. Dr. Muhammad Iliyas

Project Team:

M Taha Mukhtar BSCS51F22S004 (Team Lead)
Hammad Ahmad BSCS51F22S012 (Team Member)
Usama Farooq BSCS51F22S045 (Team Member)

Submission Date:

December 15, 2025



Project Manager's Signature

Document Information

Category	Information
Customer	UOS – Department of Computer Science
Document	Software Design Specification
Identifier	Group 40
Author(s)	Muhammad Taha Mukhtar Hammad Ahmad Usama Farooq
Project	AI powered Plant Disease Detection App (AgroGuard)
Document Version	2.0
Status	Approved
Approver(s)	PM
Issue Date	October 20, 2025

Definition of Terms, Acronyms and Abbreviations.

AI	Artificial intelligence
GPS	Global Positioning System
ML	Machine Learning
API	Application Programming Interface
MVVM	Model View View Model
HF	Hugging Face
TFLite	TensorFlow Lite

Table of Contents

1. Introduction 4
1.1 Purpose of Document 4
1.2 Project Overview 4
1.3 Scope 4
2. Design Considerations 4
2.1 Assumptions and Dependencies 5
2.2 Risks and Volatile Areas 5
3. System Architecture 5
3.1 System Level Architecture 7
3.2 Sub-System / Component / Module Level Architecture 8
3.3 Sub-Component / Sub-Module Level Architecture (1…n) 9
4. Design Strategies 11
4.1 Strategy 1…n 11
5. Detailed System Design 12
6. References 17

1. Introduction

1.1 Purpose of Document

This document provides a comprehensive design specification for the AI Plant Disease Detection mobile application. It is intended for the development team, project supervisors, and technical reviewers who will evaluate the system architecture and implementation approach. This document will serve as a blueprint for the development process, outlining the system architecture, design strategies, and technical considerations.

1.2 Project Overview

The AI Plant Disease Detection App is a mobile application designed to assist farmers in identifying plant diseases through image recognition technology. The application leverages artificial intelligence and machine learning to analyze photographs of crops and provide instant diagnosis of plant health conditions.

The system supports disease detection for multiple crop types including corn, oranges, potato, mango, lemon and apple. Upon capturing or uploading an image of a plant, the application processes the image and sends it to a remote Hugging Face-hosted inference server, where TensorFlow Lite models classify the disease. The application then provides farmers with detailed information about the identified disease, including descriptions and recommended treatments. Additionally, the app integrates weather information relevant to the farmer's location and an AI-powered agricultural chatbot powered by the Groq API (Llama 3.3 70B), helping them make informed decisions about crop management.

1.3 Scope

The system will:

- Provide a mobile application interface for capturing or uploading plant images
- Support disease detection for different crop plants such as corn, orange, potato, mango, apple, and lemon
- Send plant images to a remote TFLite inference server hosted on Hugging Face for disease classification
- Provide detailed descriptions of identified diseases
- Suggest practical treatment options for detected plant diseases
- Display real-time weather information based on the user's geographical location
- Integrate an AI-based chatbot (AgroBot) powered by Groq API for treatment and agriculture guidance

- Provide agricultural news via NewsAPI
- Support user authentication via Firebase (email/password and Google Sign-In) and profile management

The system will NOT:

- Provide a marketplace for buying/selling agricultural products
- Support real-time chat or community features between users
- Provide financial or subsidy-related services
- Include e-commerce or payment gateway integration
- Offer personalized recommendations based on user history

2. Design Considerations

2.1 Assumptions and Dependencies

Assumptions:

1. **Device Capabilities:** Target users have smartphones with functional cameras capable of capturing clear images (minimum 8MP resolution recommended)
2. **Operating System:** The application will run on Android devices with minimum Android 8.0 (API 26) OS version
3. **Image Quality:** Users will capture images in adequate lighting conditions with the affected plant part clearly visible
4. **Internet Connectivity:** An active internet connection is required for disease inference, as the TFLite models run on a remote Hugging Face server, not on the device
5. **Storage Space:** Devices have sufficient storage space (minimum 100MB) to accommodate the application; no on-device ML model is bundled

Dependencies:

1. **Remote Inference Server:** The application depends on a Hugging Face-hosted server that runs TensorFlow Lite models for plant disease diagnosis. The Android client sends plant images to the server via Retrofit API and receives diagnosis results, with no on-device model inference.
2. **AI Chatbot Service:** Groq API using the Llama 3.3 70B Versatile model for the AgroBot assistant
3. **Weather API Service:** Integration with a weather data provider for real-time weather information

4. **News API Service:** Integration with NewsAPI (newsapi.org) for fetching agricultural news
5. **Firebase Services:** Firebase Authentication, Firestore, and Firebase Storage for user management and data persistence
6. **Location Services:** Device GPS/location services must be enabled for weather data retrieval
7. **Image Processing Libraries:** ML Kit Subject Segmentation and CameraX for leaf isolation and image capture
8. **Treatment Suggestion:** Groq API using the Llama 3.3 70B Versatile model for the treatment suggestion according to the diagnose disease.

2.2 Risks and Volatile Areas

Technical Risks:

1. **Model Performance Variability:** Disease detection accuracy may vary significantly based on image quality, lighting conditions, and disease progression stages. Images captured in poor lighting or at incorrect angles may lead to misclassification.

- **Mitigation:** Implement image quality checks using ML Kit leaf segmentation, provide user guidelines for capturing images, and display confidence scores with results.

2. **Server Availability and Latency:** Since all ML inference is performed on the remote Hugging Face server, network latency or server downtime may cause slow or failed diagnosis responses.

- **Mitigation:** Implement graceful error handling and loading indicators; set appropriate OkHttp timeouts (60 seconds); inform the user clearly if the server is unreachable.

3. **API Dependency:** Weather API, NewsAPI, and Groq API services may experience downtime, or changes in terms of service.

- **Mitigation:** Implement graceful error handling, cache weather data where applicable, and design the system to allow easy switching between service providers.

Requirement-Related Risks:

1. **Scope Creep:** Stakeholders or users may request additional features (e.g., more crop types, social features) during development.

- **Mitigation:** Maintain clear scope documentation and implement a modular architecture that allows future feature additions without major refactoring.

2. **Remedy Information Accuracy:** Agricultural recommendations must be accurate and appropriate for local conditions; incorrect remedies could harm crops or be ineffective.
- **Mitigation:** Consult agricultural experts, validate remedy information with authoritative sources, and include disclaimers.

Volatile Areas (Subject to Change):

- **Supported Crop Types:** Initial scope includes six crops (corn, mango, potato, apple, orange, and lemon), but may expand based on stakeholder feedback or server-side model updates
- **Weather Information Display:** The extent and format of weather data presentation may evolve based on user feedback
- **Disease Database:** Disease descriptions and remedies may require updates as new agricultural research becomes available
- **ML Model Updates:** The disease detection TFLite models are hosted on Hugging Face and can be updated server-side without requiring an app update
- **User Interface Design:** UI/UX may undergo iterations based on usability testing with actual farmers

Contingency Plans:

- Design modular architecture to accommodate new features without major rewrites
- Implement version control for ML models on the Hugging Face server to allow rollback if new versions perform poorly
- Document all external dependencies and maintain alternative options where feasible

3. System Architecture

3.1 System Level Architecture

Agro Guard is a native Android mobile application built with Kotlin and Jetpack Compose that helps farmers identify plant diseases through AI-powered image analysis. The system uses the MVVM (Model-View-ViewModel) architectural pattern with Firebase for backend services and a remote Hugging Face server for ML inference.

Core Components:

- **Android Application Layer:** UI (Jetpack Compose), Presentation (ViewModel/StateFlow), Data (Repositories and ViewModels).
- **Backend Services:** Firebase (Auth, Firestore, Storage), Remote TFLite Inference API (Hugging Face Space – Python backend), AI Chat Service (Groq API – Llama 3.3), News API (newsapi.org), Weather API.
- **Cloud Services:** Firebase Platform (Auth, Firestore).
- **Local Storage:** Not applicable for ML models; no on-device model inference. Firebase SDK local cache handles offline Firestore reads.

Key Subsystems:

- **Authentication System:** Firebase Authentication (Email/Password, Google Sign-In).
- **UI Subsystem:** Jetpack Compose (Declarative UI), Navigation Compose, bottom navigation bar with Home, Diagnose, Assistant, and Profile tabs.
- **Image Processing:** CameraX (image capture), ML Kit Subject Segmentation (leaf isolation), Coil (image loading), Coroutines (async processing).
- **ML Inference:** Disease detection via remote REST API hosted on Hugging Face Space (Retrofit + OkHttp, multipart image upload). No on-device TFLite inference.
- **AI Chatbot:** AgroBot assistant powered by Groq API using Llama 3.3 Versatile model, accessed via Retrofit.
- **News Feed:** Agricultural news fetched from NewsAPI via Retrofit, displayed in NewsScreen.
- **Weather:** Real-time weather data fetched using device GPS location via WeatherViewModel.

High-Level Architecture Diagram

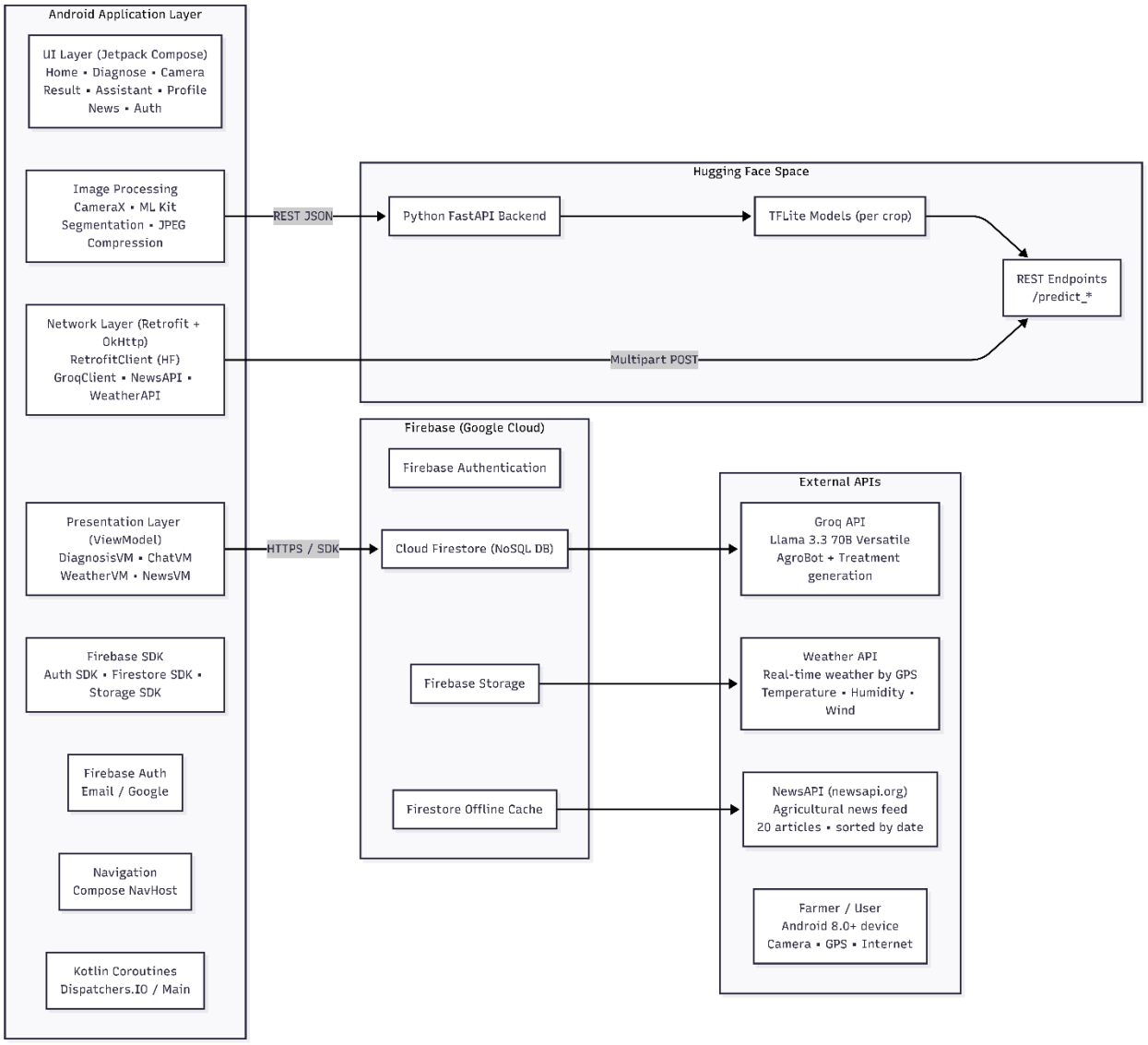


Figure 3.1 – System Level Architecture Diagram

3.2 Sub-System / Component / Module Level Architecture

This section breaks down the major components of the Agro Guard system and their interactions using modern Android standards.

Mobile Application Components

The Android application follows Google's Guide to App Architecture:

1. **UI Layer (Jetpack Compose):**
    - **LoginScreen, SignUpScreen**: Authentication screens with email/password and Google Sign-In
    - **HomeScreen:** Weather display, quick actions (AgroBot, News, Camera launch)
    - **DiagnoseScreen:** Crop selection (Corn, Tomato, Potato, Apple, Orange) and image capture/upload
    - **CameraScreen:** CameraX-based photo capture
    - **ResultScreen:** Displays disease name, confidence score, and Groq-generated treatment advice
    - **AssistantScreen:** AgroBot AI chatbot UI
    - **NewsScreen:** Agricultural news feed
    - **ProfileScreen:** EditProfileScreen, FAQScreen — User profile management

2. **Presentation Layer (ViewModel):**
    - **DiagnosisViewModel:** Handles image processing (ML Kit leaf segmentation), multipart upload to Hugging Face API, and Groq API call for treatment text.
    - **ChatViewModel:** Manages AgroBot conversation state and Groq API calls.
    - **WeatherViewModel:** Fetches and exposes weather data based on device location.
    - **NewsViewModel:** Fetches and exposes agricultural news from NewsAPI.

3. **Data Layer (Network Clients):**
    - **RetrofitClient:** Singleton Retrofit instance targeting the Hugging Face inference server with 60-second timeouts.
    - **GroqClient:** Singleton Retrofit instance targeting api.groq.com for AI chatbot and treatment generation.
    - **GroqApiService:** Defines POST call to /openai/v1/chat/completions with the Llama 3.3 70B model.

Component Interaction Diagram (MVVM)

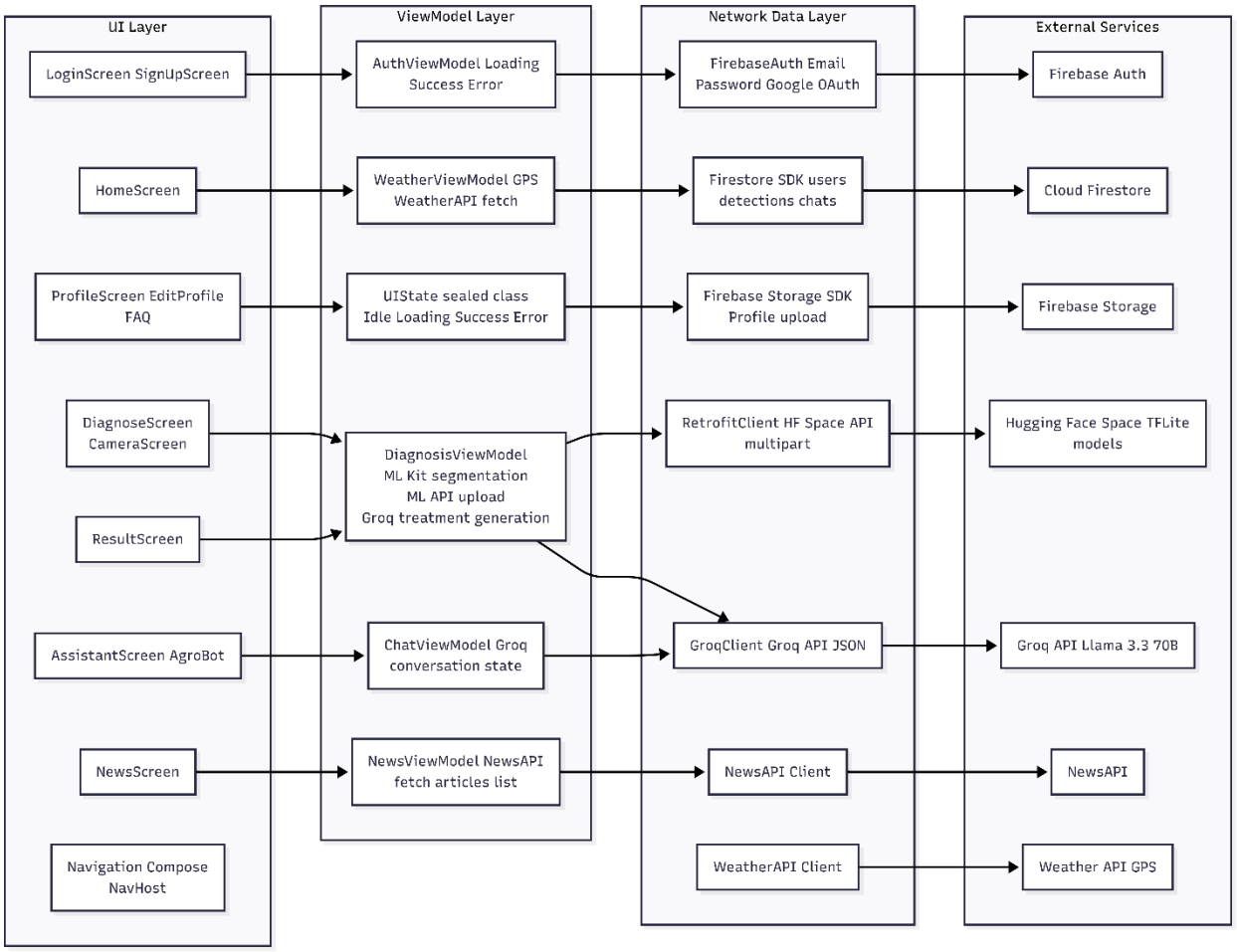


Figure 3.2 – MVVM Component Interaction Diagram

3.3 Sub-Component / Sub-Module Level Architecture (1…n)

3.3.1 Authentication System (Firebase)

The system uses Firebase Authentication for secure identity management.

- **Registration:** Full name, email, and password fields. Email/password validation handled in SignUpScreen. Upon successful registration, a corresponding user document is created in the users collection in Firestore.
- **Google Sign-In:** Integrated via GoogleSignIn SDK and FirebaseAuth.GoogleAuthProvider. Launched from both LoginScreen and SignUpScreen.

- **Session:** Managed automatically by the Firebase SDK. FirebaseAuth.getInstance().currentUser persists across app restarts.
- **Token Management:** Firebase handles token refresh automatically.

3.3.2 ML Inference Engine (Remote – Hugging Face)

- **Primary Inference:** The Android app processes the image locally using ML Kit Subject Segmentation to isolate the leaf, compresses it to JPEG, then sends it as a multipart HTTP POST to the Hugging Face-hosted server. The server runs the appropriate TFLite model (selected based on crop type) and returns a JSON response containing the disease name and confidence score.
- **Crop Endpoints:** The ApiService uses a dynamic @Path parameter to route requests to the correct endpoint.
- **On-Device Inference:** Not implemented. All model inference is performed server-side. The org.tensorflow:tensorflow-lite library is NOT a dependency of this application.
- **Treatment Generation:** After receiving the diagnosis from the Hugging Face server, the DiagnosisViewModel calls the Groq API to generate tailored treatment advice, which is displayed on the ResultScreen.

3.3.3 API Integration Manager (Retrofit)

All non-Firebase network calls are handled by Retrofit with OkHttp.

- **RetrofitClient:** Targets the Hugging Face inference server. Handles multipart image uploads for disease detection.
- **GroqClient:** Targets api.groq.com. Handles JSON chat completion requests for both AgroBot conversations and treatment text generation.
- **Timeouts:** Both clients are configured with 60-second connect, read, and write timeouts to accommodate server processing time.
- **Error Handling:** try-catch blocks within Coroutines handle IOExceptions and HttpExceptions, surfacing errors to the UI via UiState.Error.

3.3.4 Image Processing Pipeline (Kotlin/Coroutines)

1. **Capture:** Uses CameraX API for lifecycle-aware camera control (CameraScreen).
2. **Selection:** Uses ActivityResultContracts.PickVisualMedia for secure gallery access.
3. **Leaf Isolation:** ML Kit Subject Segmentation (FallbackGreenMask as fallback) is applied to remove the background and isolate the leaf (LeafBackgroundRemover / ImageProcessor utility classes).

4. **Compression:** Images are compressed to JPEG before being sent as multipart body to the API.
5. **Asynchronous:** All image operations run on Dispatchers.IO to prevent UI freezing.

3.3.5 Database Schema (Firestore)

Firestore is a NoSQL document database. The schema is optimized for reads.

Collections:

- **users (Collection)**
  - **Document ID:** uid (from Firebase Auth)
  - **Fields:** email, name, createdAt, profileImageUrl
- **detections (Collection)**
  - **Document ID:** Auto-generated
  - **Fields:** userId (index), imageUrl, diseaseName, confidence, timestamp, plantType
- **chats (Collection)**
  - **Document ID:** Auto-generated
  - **Fields:** userId, message, sender ("user" or "bot"), timestamp

4. Design Strategies

4.1 Strategy 1…n

4.1.1 Authentication Strategy

- **Implementation:** Firebase Authentication (Email/Password + Google Sign-In).
- **Why:** Reduces backend boilerplate, provides free tier Email auth and Google OAuth, and integrates seamlessly with Firestore security rules.

4.1.2 Remote ML Inference Strategy

- **Implementation:** TFLite models are hosted and served from a Hugging Face Space (Python backend). The Android app sends images via Retrofit multipart POST and receives JSON diagnosis results.

- **Why:** Eliminates the need to bundle large ML model files in the APK, allows server-side model updates without requiring app updates, and offloads computation from the device.

4.1.3 AI Chatbot Strategy (Groq API)

- **Implementation:** AgroBot is powered by the Groq API using the Llama 3.3 70B Versatile model. Requests are made via GroqClient (Retrofit) with conversation history passed in each call.
- **Why:** Provides fast inference for agricultural Q&A without managing an LLM server. The same API is also used to generate treatment recommendations after disease detection.

4.1.4 MVVM Architecture

- **Objective:** Separation of concerns and testability.
- **Async:** Kotlin Coroutines replace RxJava for handling asynchronous operations including API calls, image processing, and Firebase reads/writes.
- **State Management:** UiState sealed class (Idle, Loading, Success, Error) drives UI rendering in Compose.

4.1.5 Migration to Native Android (Kotlin)

- **Performance:** Native code runs faster than JS bridge solutions, especially for image processing.
- **Camera:** CameraX offers more stable and device-agnostic control over hardware.
- **UI:** Jetpack Compose offers a modern, declarative UI toolkit that is type-safe and compiled.

4.1.6 Deployment Architecture

The Android app is distributed via Google Play Console. The backend is split between Firebase (Serverless) and the Hugging Face Space (ML inference server).

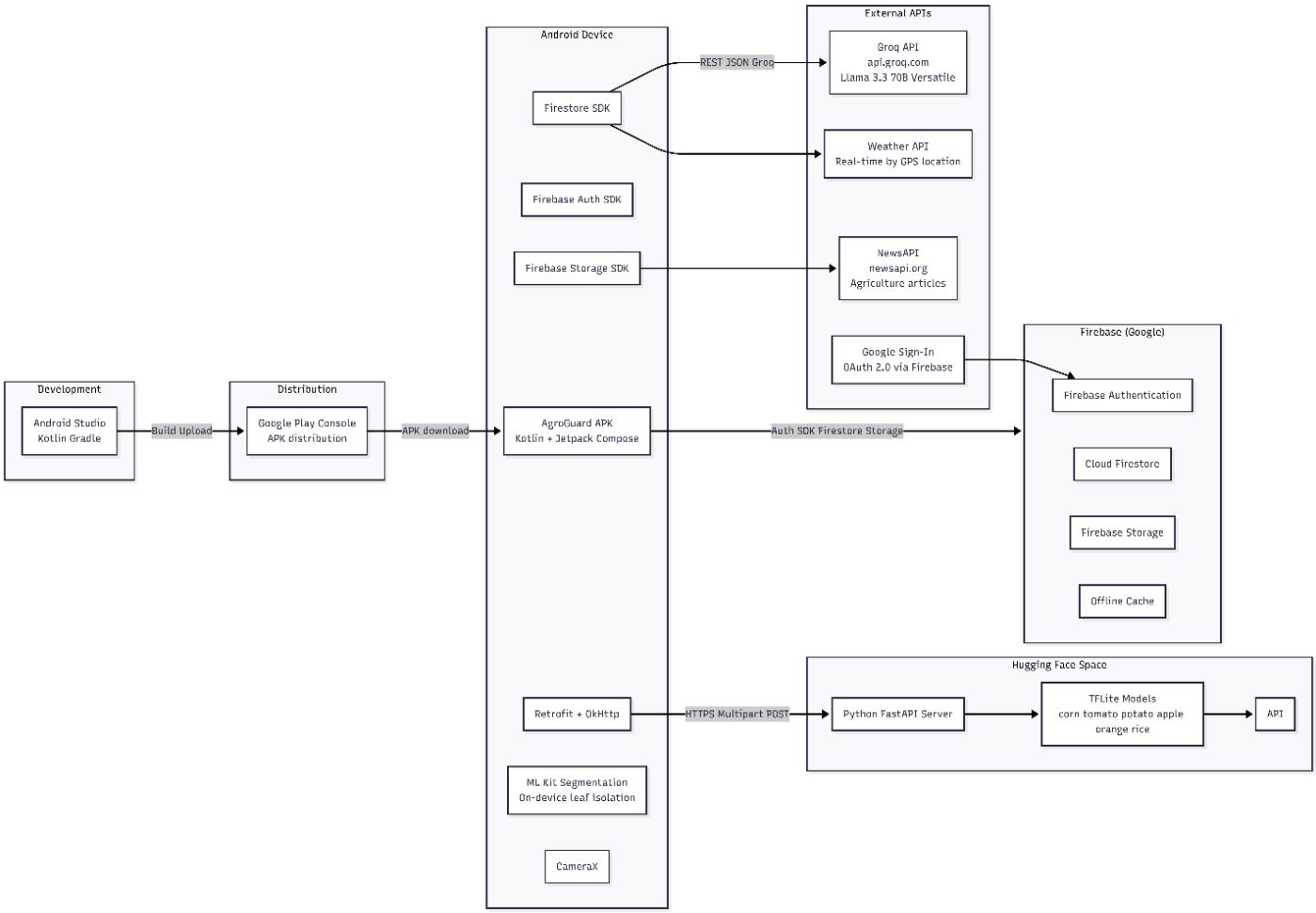


Figure 4.1.6 – Deployment Architecture Diagram

5. Detailed System Design

The detailed system design provides a comprehensive architectural blueprint of the AI Assistant for Plant Disease Diagnosis, presenting multiple design perspectives that define system structure, behavior, and data management.

- **Class Diagram:** The system defines classes across multiple modules including User Management, Disease Detection, ChatBot, Weather Services, News, and Location. Each class specifies relevant attributes and methods, while UML relationships illustrate interactions and dependencies between system components.
- **Sequence Diagram:** The sequence diagrams illustrate six core system workflows: Registration, Login, Weather Display, Disease Detection, ChatBot Interaction, and Logout. Each diagram presents step-by-step message flows between the interacting system components.
- **State Transition Diagram:** The state transition diagram maps the application states and user navigation flow from launch through various features, highlighting decision points, loop-back states, and error-handling transitions.
- **Logical Data Model:** The logical data model presents the conceptual data structure of the system, identifying key entities such as User, Disease, Treatment, and DetectionResult, along with the relationships among them.
- **Physical Data Model:** The physical data model defines the actual database implementation using Firestore (NoSQL), specifying collections, fields, and relationships.
- **Detailed GUIs:** The detailed GUI designs provide visual representations of all major application screens, ensuring a consistent, intuitive, and user-friendly experience.

5.1 Class Diagram:

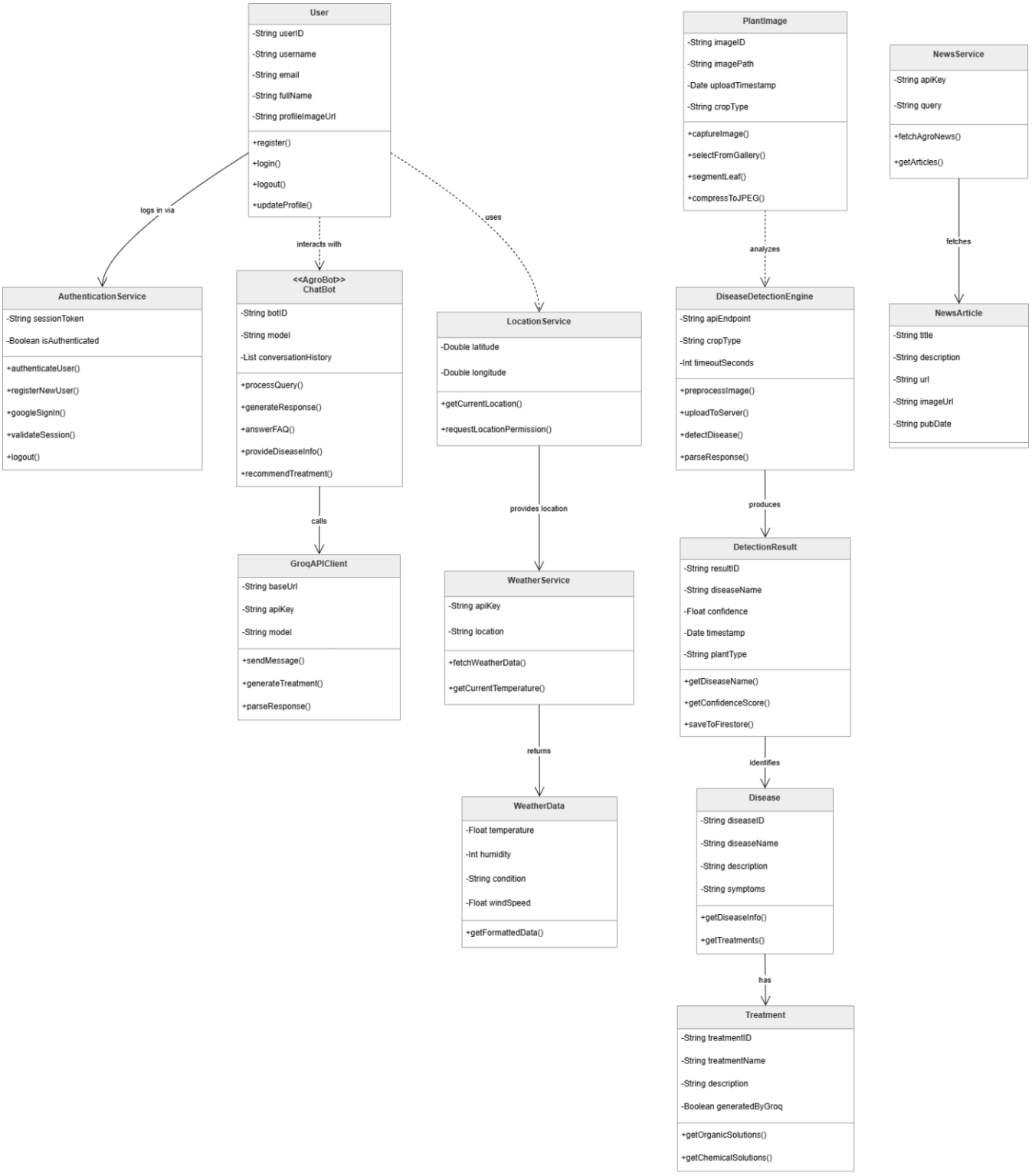


Figure 5.1 – Class Diagram

5.2 Sequence Diagram:

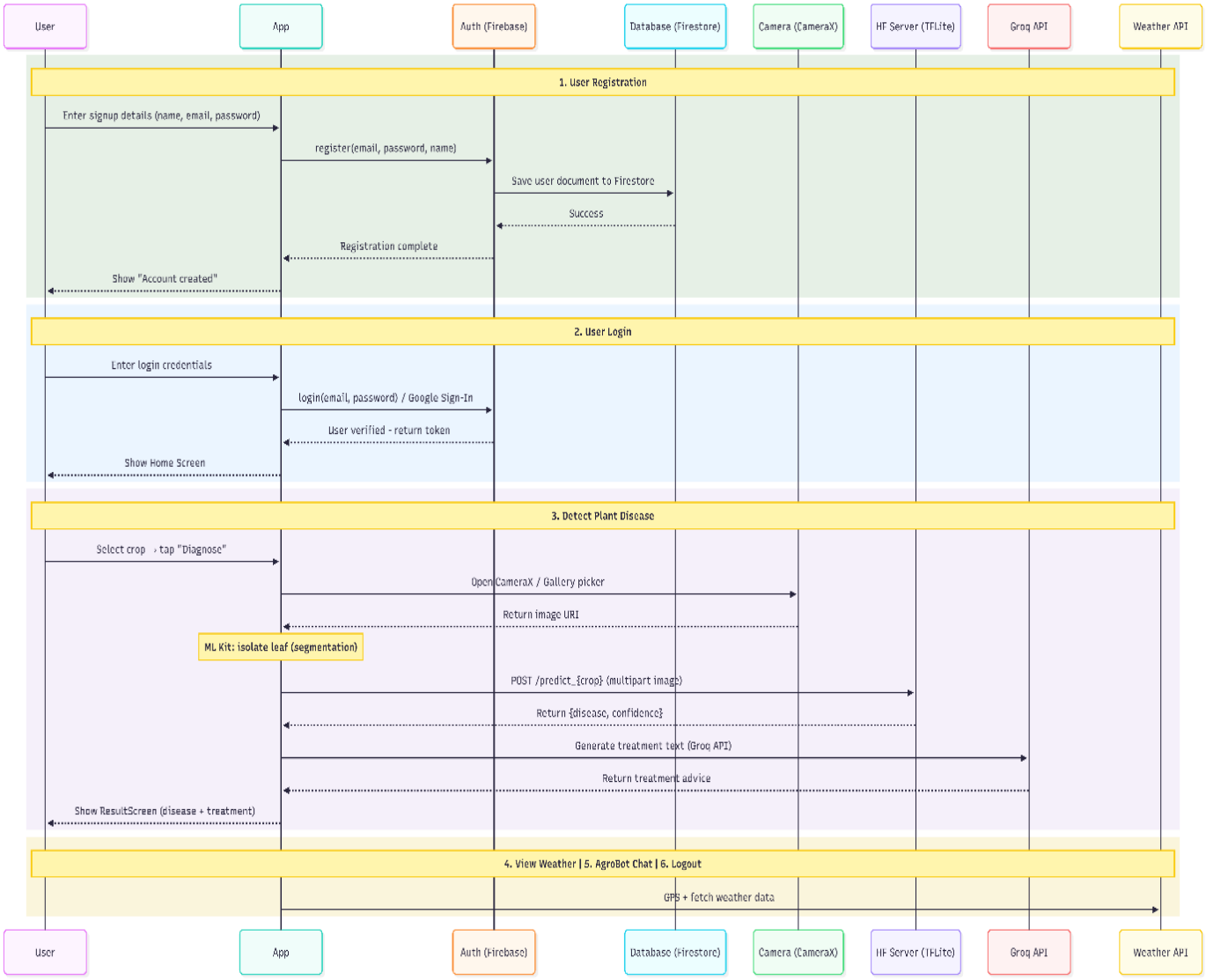


Figure 5.2 – Sequence Diagram

5.3 State Transition Diagram:

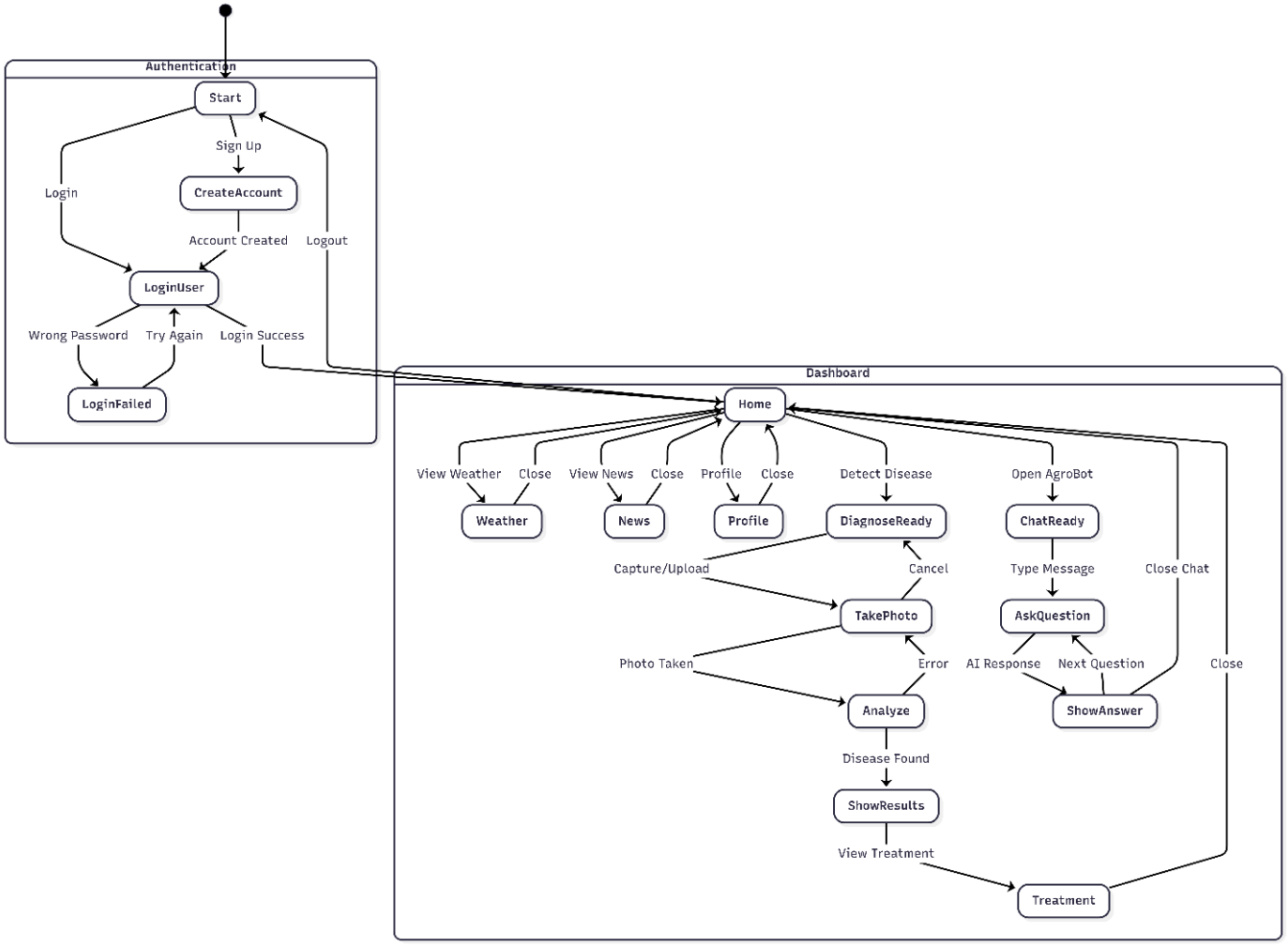


Figure 5.3 – State Transition Diagram

5.4 E/R Diagram:

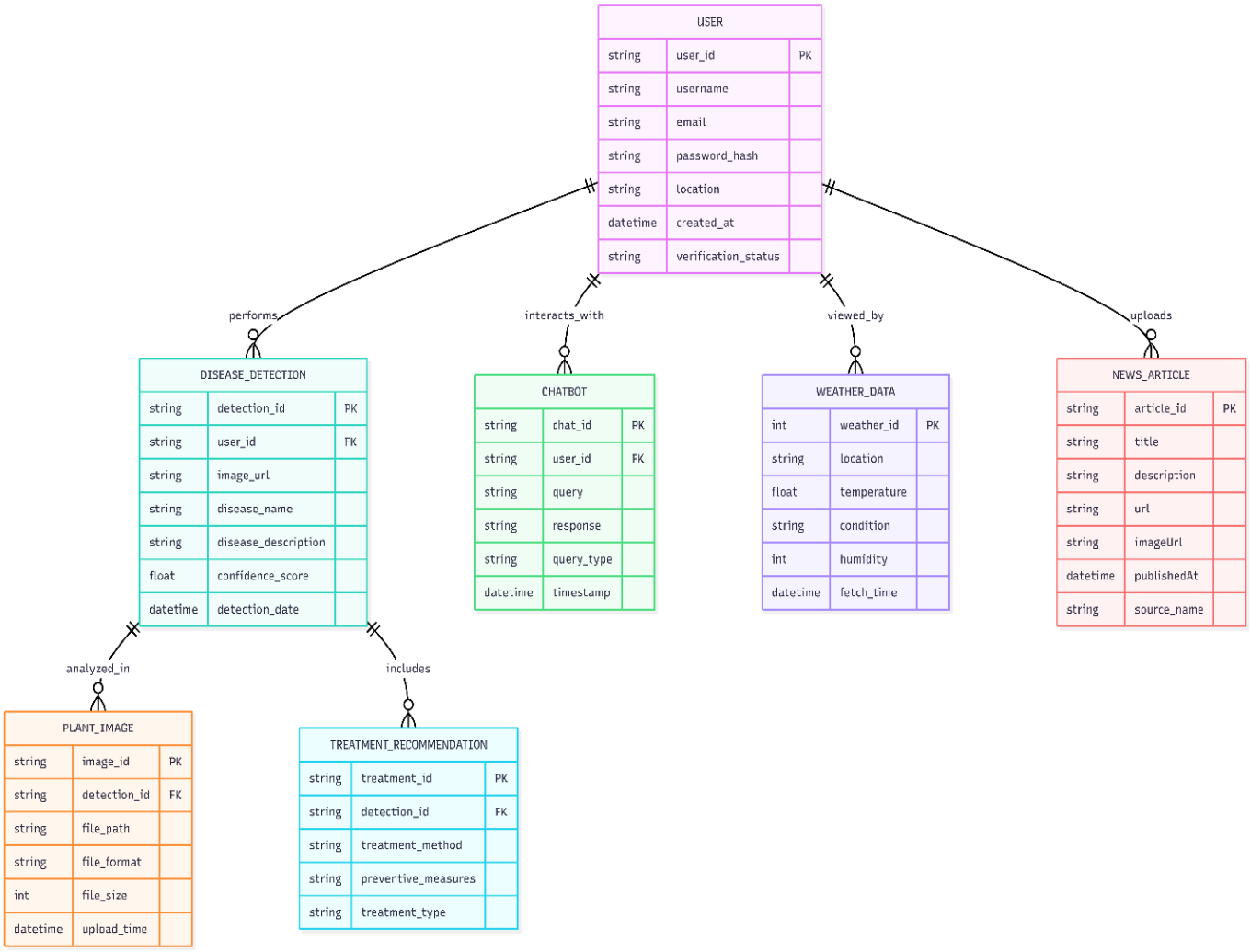
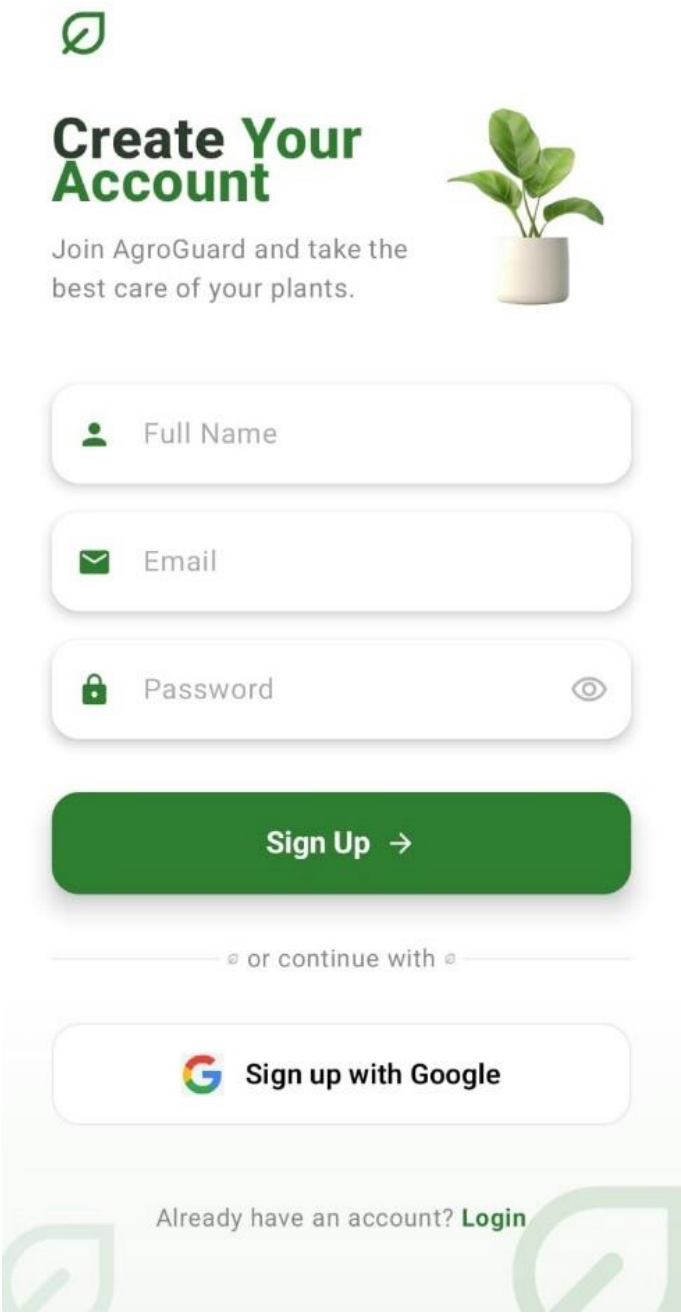
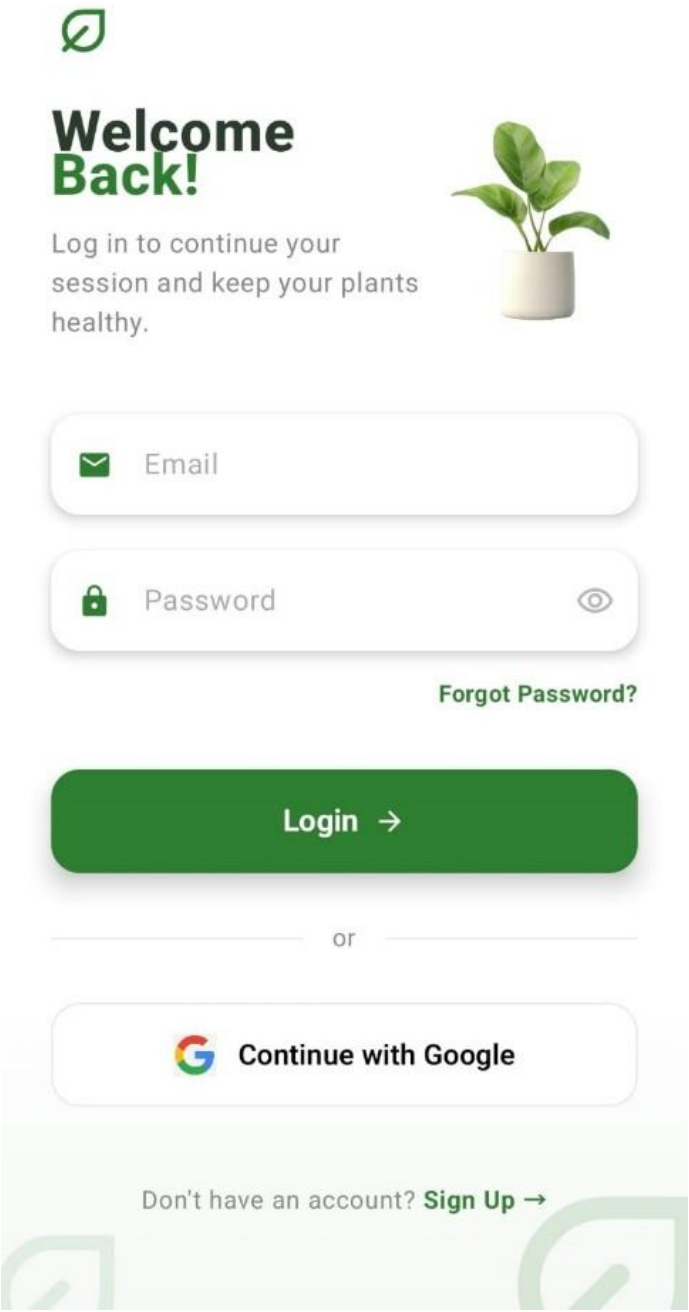


Figure 5.4 – E/R Diagram

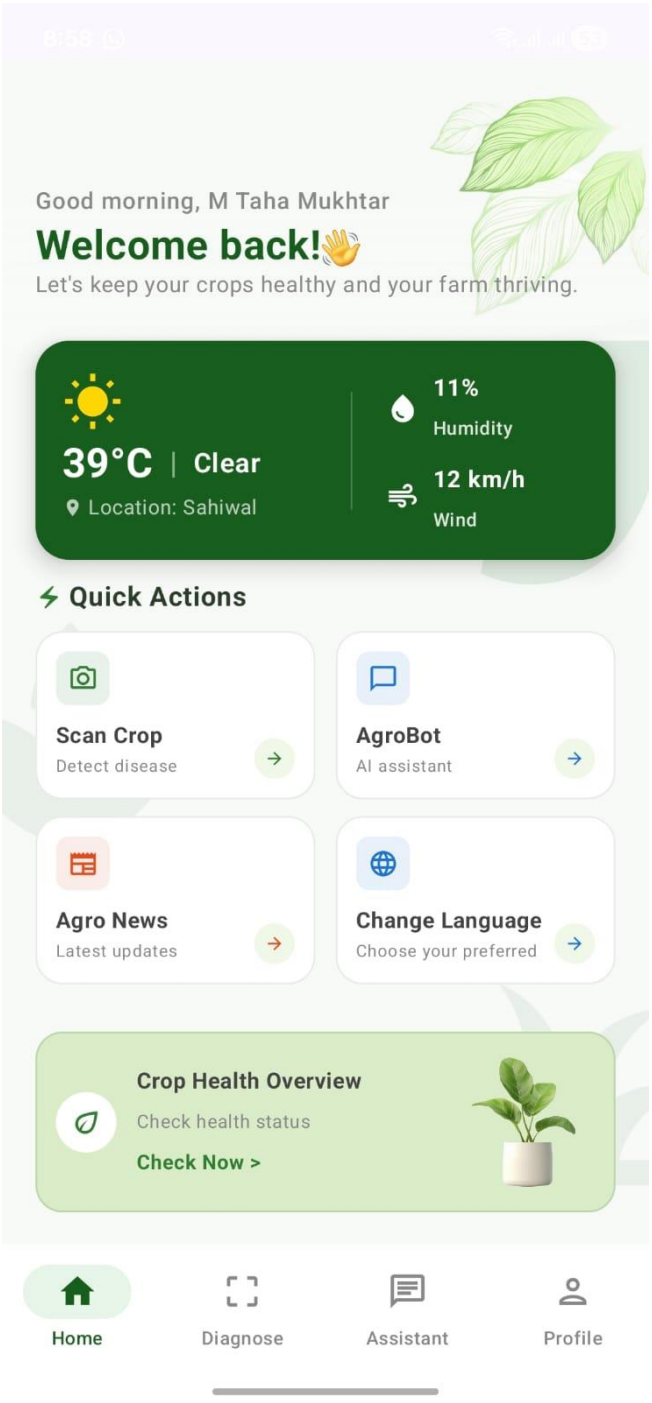
5.5 Graphical User Interface:

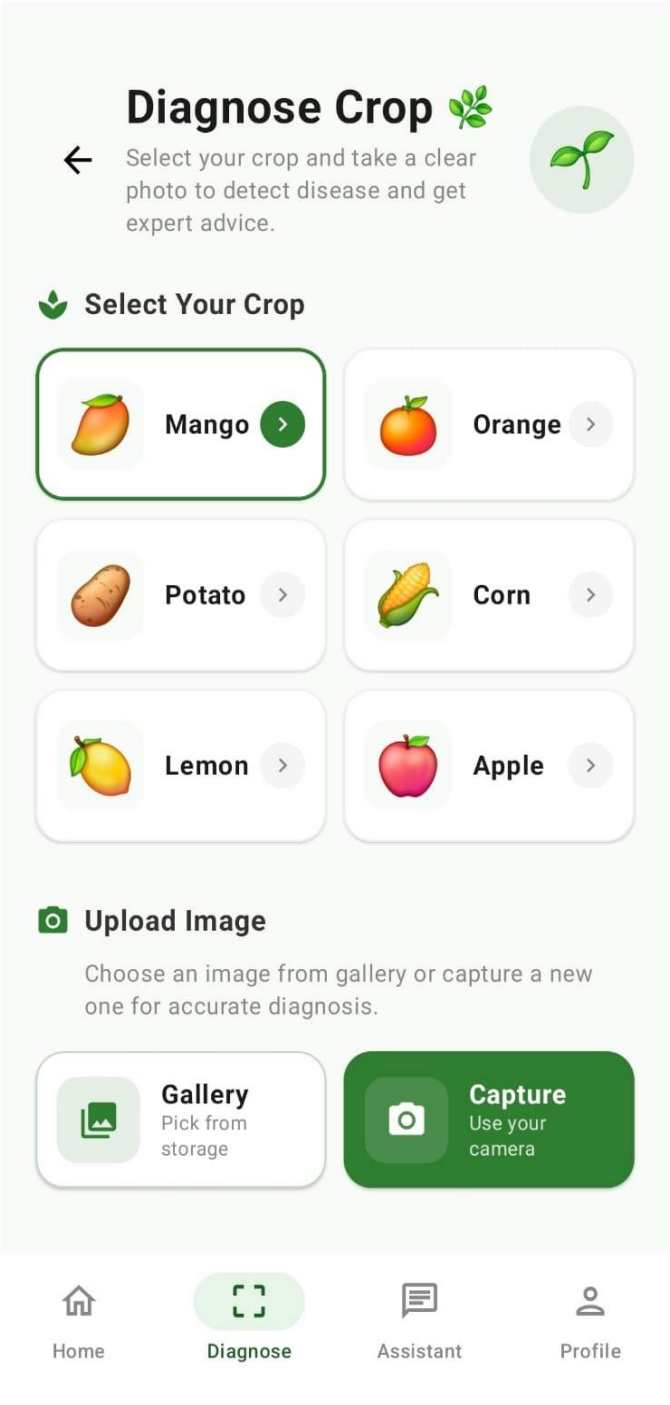




A. (Account Creation Screen)

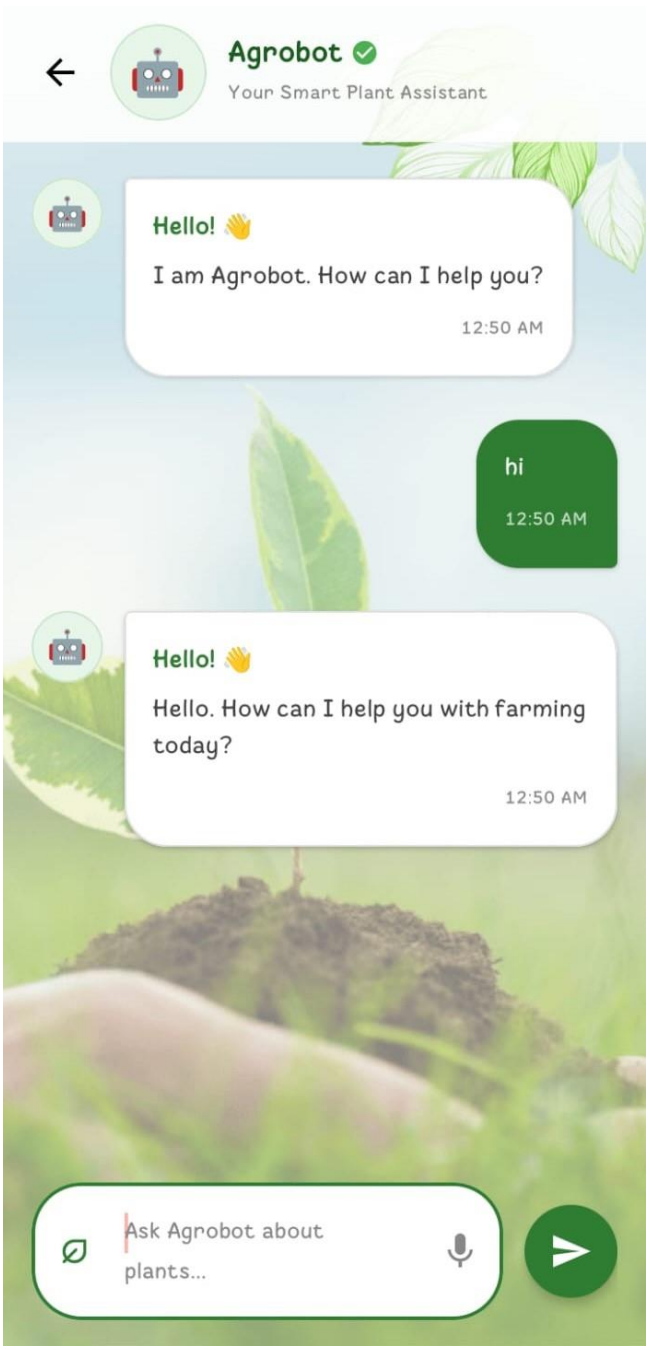
B. (Login Scree)



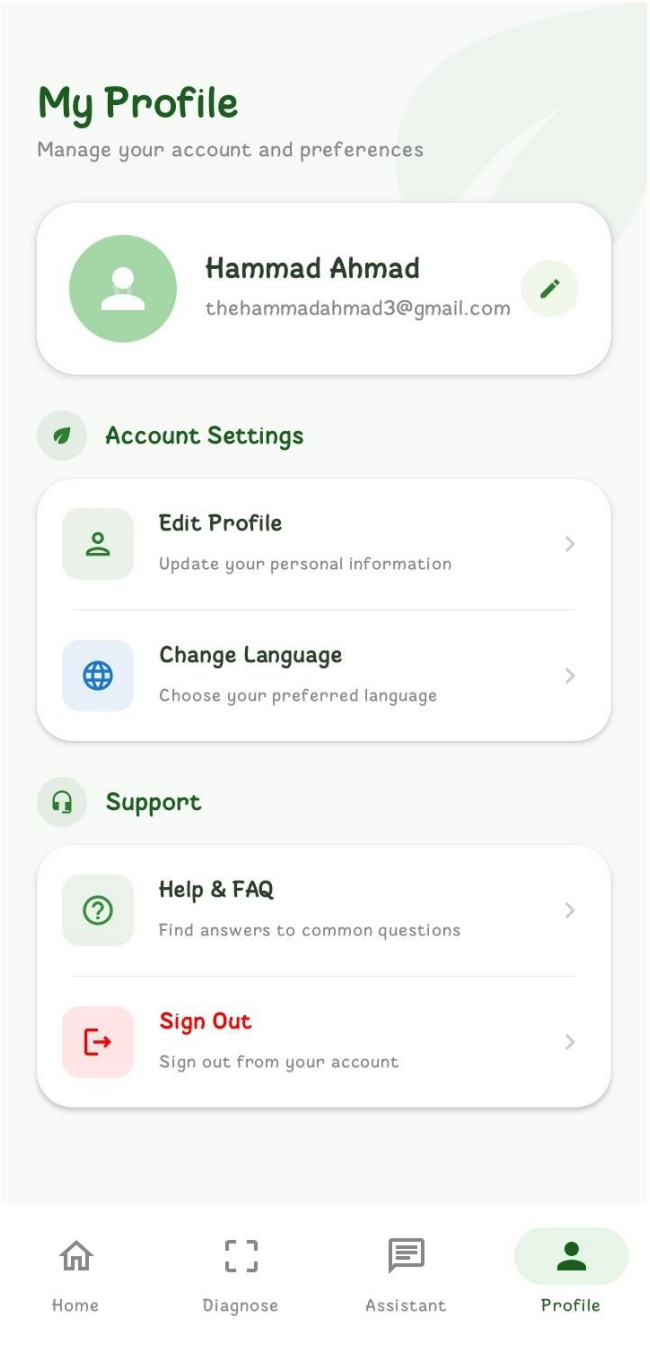


C. (Home Screen)

D. (Diagnosis Screen)



E. (Assistant Screen)



F. (Profile Screen)

6. References

Ref. No.	Document Title	Date of Release/ Publication	Document Source
Group40	Project Proposal	Oct 10, 2025	https://github.com/mtahamukhtar2-ui/FYP/...
Group40	SRS	Oct 20, 2025	https://github.com/mtahamukhtar2-ui/FYP/...
Group40	Diagrams	Dec 15, 2025	https://github.com/mtahamukhtar3-star/FInal-Diagrams.git